Université de Lille
Licence MIASHS – Parcours MEF

Travail d'Étude et de Recherche

Méthodes algorithmiques fondées sur les graphes pour l'analyse et l'interprétation de données génomiques

Evgenii Kniazev

Année universitaire 2025–2026

Encadrant : Charles Paperman

Ce rapport étudie si les Roaring Bitmaps constituent une alternative pertinente à l'index de k-mers de Vizitig, un outil permettant de construire, coloriser et interroger des graphes de De Bruijn colorés. L'implémentation actuelle repose sur un fichier binaire trié projeté en mémoire (*memory-mapped*), exploité par une fusion triée en $O(n+m)$ pour la colorisation – opération qui devient un goulot d'étranglement dans un contexte pangenomique. Nous formulons l'hypothèse que les Roaring Bitmaps 64 bits (RoaringTreemap) pourraient améliorer les performances d'appartenance et d'intersection.
L'analyse de densité montre cependant que l'encodage 2 bits des k-mers les disperse dans un univers de $4{,}6 \times 10^{18}$ valeurs, de sorte que les *array containers* dominent universellement et que l'avantage SIMD de Roaring (ET bit-à-bit sur les *bitmap containers*) ne s'applique pas. Par ailleurs, la jointure présente un défi structurel : un bitmap Roaring ne stocke pas les `node_id` associés aux k-mers, nécessitant une étape de récupération supplémentaire. La comparaison de performances reste donc une question empirique.

Table des matières

1 Contexte **2**
1.1 Génomique 2
1.2 Colorisation des graphes 4
1.3 Les trois opérations fondamentales 5
1.4 L'approche de Vizitig 5

2 Roaring bitmaps **6**
2.1 Approches précédentes de compression 6
2.2 Principe des bitmaps 8
2.3 Structure des Roaring bitmaps 8

3 Application à Vizitig **9**
3.1 L'index de k-mers actuel 9
3.2 Analyse de la densité 10
3.3 Analyse des opérations fondamentales 11
3.4 Proposition 12

4 Conclusion **12**

1 Contexte

1.1 Génomique

La génomique étudie la structure et la fonction des génomes. Un génome est une séquence de nucléotides – adénine (A), thymine (T), guanine (G) et cytosine (C) – dont la longueur peut atteindre plusieurs milliards de lettres chez les eucaryotes (animaux, plantes). Or, les technologies de séquençage les plus répandues ne sont pas conçues pour lire une telle séquence d'une seule traite : elles produisent un grand nombre de lectures courtes, typiquement d'une centaine de nucléotides [1], qu'il faut ensuite assembler pour reconstruire la séquence d'origine.

Pour illustrer le problème, reprenons l'exemple de Compeau et al. [2] représenté en figure 1. On considère un petit génome circulaire, `ATGGCGTGCA`, à partir duquel cinq lectures très courtes ont été produites : `CGTGCAA`, `ATGGCGT`, `CAATGGC`, `GGCGTGC` et `TGCAATG`. L'objectif est de reconstruire le génome d'origine à partir de ces seules lectures, sans connaissance préalable de sa structure.

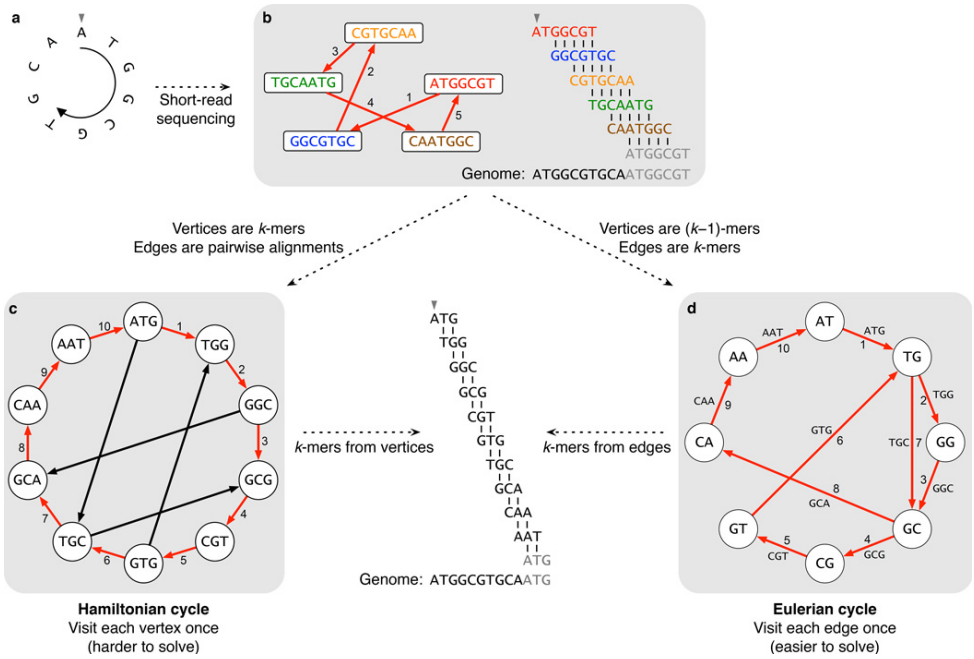

FIGURE 1 – Exemple d'assemblage à partir d'un petit génome circulaire `ATGGCGTGCA` et de cinq lectures courtes. Figure tirée de Compeau et al. [2].

Une première approche consiste à repérer les chevauchements entre lectures : deux fragments qui partagent un suffixe et un préfixe identiques proviennent vraisemblablement de régions voisines du génome. Dans notre exemple, `ATGGCGT` et `GGCGTGC` se chevauchent sur cinq nucléotides (`GGCGT`), ce qui suggère qu'ils doivent être concaténés dans la reconstruction. En répétant ce raisonnement pour toutes les paires de lectures, on obtient un ensemble de relations de chevauchement qu'il s'agit ensuite d'ordonner : il faut trouver

une séquence des lectures telle que chacune soit utilisée exactement une fois et que deux lectures consécutives se chevauchent effectivement.

Cette situation se prête naturellement à une représentation sous forme de *graphe* : chaque lecture devient un sommet, et un chevauchement suffisant entre deux lectures une arête orientée reliant l'une à l'autre. Le problème de reconstruction se reformule alors comme la recherche d'un parcours qui visite chaque sommet exactement une fois – ce qu'on appelle un *chemin hamiltonien*.

Cette approche, bien qu'intuitive, se heurte rapidement à deux difficultés. D'une part, les lectures contiennent des erreurs de séquençage, ce qui complique la détection de chevauchements exacts. D'autre part, le calcul des chevauchements entre toutes les paires de lectures devient prohibitif à grande échelle : un séquenceur Illumina produit couramment plusieurs millions de lectures par exécution, ce qui conduirait à un nombre d'alignements deux-à-deux de l'ordre de 10^{12} [2]. Pire encore, le problème du chemin hamiltonien est NP-complet, et donc computationnellement difficile à résoudre sur de grands graphes.

L'article fondateur de Pevzner, Tang et Waterman [3] a montré qu'il était possible de reformuler l'assemblage comme la recherche d'un *chemin eulérien* – c'est-à-dire un parcours qui emprunte chaque arête exactement une fois – en s'appuyant sur les graphes introduits par Nicolaas de Bruijn en 1946. Ce changement de perspective est décisif : contrairement au cas hamiltonien, la recherche d'un chemin eulérien s'effectue en temps polynomial.

Le principe est le suivant. On fixe une longueur k, appelée taille de *k-mer*, et l'on découpe chaque lecture en toutes ses sous-chaînes consécutives de longueur k. Les sommets du graphe correspondent alors aux $(k-1)$-mers distincts, et chaque k-mer est représenté par une arête orientée reliant son préfixe de longueur $k-1$ à son suffixe de même longueur.

Appliquons cette construction à notre exemple avec $k = 3$. Chaque lecture est décomposée en 3-mers : `ATGGCGT` donne `ATG`, `TGG`, `GGC`, `GCG` et `CGT` ; `GGCGTGC` donne `GGC`, `GCG`, `CGT`, `GTG` et `TGC` ; et ainsi de suite pour les trois autres lectures. Les sommets du graphe sont les 2-mers distincts (`AT`, `TG`, `GG`, `GC`, `CG`, `GT`, `CA`, `AA`), et chaque 3-mer devient une arête : `ATG` relie `AT` à `TG`, `TGG` relie `TG` à `GG`, et ainsi de suite. La séquence génomique d'origine se retrouve alors en parcourant un cycle eulérien dans ce graphe – chaque arête traversée contribuant d'un nucléotide à la reconstruction.

Cette construction capture efficacement les chevauchements entre fragments tout en ramenant l'assemblage à un problème polynomial. Elle constitue aujourd'hui la base de la plupart des assembleurs modernes, notamment BCALM 2 [4] et Cuttlefish 2 [5], qui produisent des graphes de De Bruijn compactés à partir de très grandes collections de lectures.

1.2 Colorisation des graphes

Lors de la construction du graphe de De Bruijn, on obtient une topologie – la structure des chevauchements entre k-mers – mais l'origine de chaque sommet reste indéterminée. Des algorithmes d'annotation permettent de pallier cette lacune en associant, à chaque élément du graphe, les échantillons biologiques dont il provient. Ce processus est appelé *colorisation* : chaque source biologique reçoit une *couleur*, représentée par un identifiant entier.

Dans les graphes de De Bruijn colorés, chaque couleur correspond à un échantillon biologique distinct. À chaque sommet du graphe est associé l'ensemble des couleurs des échantillons qui contiennent ce k-mer. Cette correspondance permet la détection de variations génétiques [8] : si un k-mer est présent dans un échantillon malade mais absent d'un échantillon sain, son ensemble de couleurs encode directement cette différence, fournissant un signal de variation à l'échelle du k-mer.

D'un point de vue algorithmique, cette étape repose sur la manipulation d'ensembles de couleurs, ce qui nécessite de pouvoir effectuer efficacement des opérations ensemblistes telles que l'union, l'intersection et la différence, ainsi que des tests d'appartenance. Sans cette information de couleur, on ne disposerait que de la structure topologique du graphe : bien que celle-ci permette l'assemblage, elle ne fournit pas d'indications sur l'origine des fragments, ce qui limite fortement l'analyse biologique.

L'introduction de ces ensembles de couleurs pose d'importants défis en termes de stockage et de performances. Almodaresi et al. [9] ont montré en 2018 que les index classiques de graphes de De Bruijn compactés, fondés sur des tables de hachage reliant chaque k-mer à son unipath et à sa position, entraînaient une consommation mémoire considérable : un tel index pour le génome humain pouvait nécessiter entre 40 et 100 Go de RAM.

Les premières approches de graphes colorés illustrent une problématique encore plus importante. Muggli et al. [10] rapportent qu'en 2017, la méthode introduite par Simpson et al. dans l'assembleur ABySS stockait le graphe coloré sous forme de table de hachage distribuée, nécessitant jusqu'à 336 Go de mémoire pour un ensemble de lectures provenant d'un génome humain.

Afin de réduire cette empreinte mémoire, des représentations plus succinctes ont été proposées. Muggli et al. s'appuient sur la représentation BOSS introduite par Bowe et al. [11], qui fournit une version compressée du graphe de De Bruijn et sert de base à des structures de graphes colorés plus compactes. Ces travaux mettent en évidence un compromis fondamental entre expressivité, rapidité des requêtes et consommation mémoire, soulignant l'importance du choix des structures de données.

1.3 Les trois opérations fondamentales

La colorisation d'un graphe de De Bruijn repose sur trois opérations élémentaires appliquées à des ensembles de k-mers.

— **Appartenance** : étant donné un k-mer, déterminer s'il est présent dans l'index du graphe. Cette opération est notamment utilisée lors des requêtes `Kmer(seq)`.

— **Intersection** : étant donné l'ensemble des k-mers du graphe et l'ensemble des k-mers d'un échantillon biologique, identifier les k-mers qu'ils ont en commun. C'est le cœur de l'étape de colorisation : les k-mers partagés permettent de rattacher les nœuds du graphe à l'échantillon.

— **Jointure** : pour chaque k-mer identifié lors de l'intersection, récupérer la donnée associée – typiquement l'identifiant du nœud correspondant dans le graphe – afin de construire les assignations de couleurs.

Ces trois opérations sont indépendantes du choix de la structure de données sous-jacente ; c'est précisément ce choix que ce travail cherche à optimiser.

1.4 L'approche de Vizitig

Vizitig est un outil permettant de construire, coloriser et interroger des graphes de De Bruijn colorés [6]. En entrée, il reçoit d'une part un graphe compacté produit par des outils tels que BCALM 2 [4] ou GGCAT, et d'autre part un fichier FASTA par échantillon biologique. En sortie, il fournit un graphe coloré interrogeable via des expressions booléennes (`Color(s1) AND Color(s2)`, `OR`, `NOT`, `Kmer(seq)`, etc.).

La couche d'indexation des k-mers est assurée par **Vizibridge**, une bibliothèque compilée en Rust exposée à Python via PyO3. Vizibridge s'appuie sur **vizicomp**, un crate Rust privé qui fournit la structure `Index<K,V>` : un fichier binaire trié projeté en mémoire (*memory-mapped*), représentant un tableau statique de paires `IndexEntry{key: K, val: V}`. Dans le cas concret de Vizitig, $k = 31$ est la valeur par défaut : cette longueur est impaire (ce qui évite les k-mers palindromiques), et son encodage 2 bits tient dans 62 bits, permettant de stocker chaque k-mer dans un `u64`. Chaque index est alors un fichier de paires (k-mer_{u64}, node_id_{u64}) trié par k-mer, où le `node_id` identifie l'unitig (nœud du graphe compacté) auquel appartient le k-mer. L'index est réparti en fragments (*shards*) pour permettre le traitement parallèle. Sur les architectures x86_64, Vizibridge détecte et utilise automatiquement les extensions SIMD disponibles (AVX2, AVX512) pour accélérer ces opérations.

Cette implémentation prend en charge les trois opérations introduites précédemment, mais sa performance aux grandes échelles reste une question ouverte. Les Roaring Bitmaps constituent une alternative prometteuse : apparus en 2016 [7], ils sont aujourd'hui intégrés dans de nombreux systèmes de production – Apache Lucene, Apache Druid, le moteur SQL de YouTube (Google Procella) – et ont démontré des gains de performance significatifs

pour des opérations ensemblistes sur des identifiants entiers – dans des domaines denses, dont la pertinence pour les k-mers sera examinée en section 3.2. La section 2 présente leur fonctionnement, et la section 3 examine si leur application à Vizitig est pertinente.

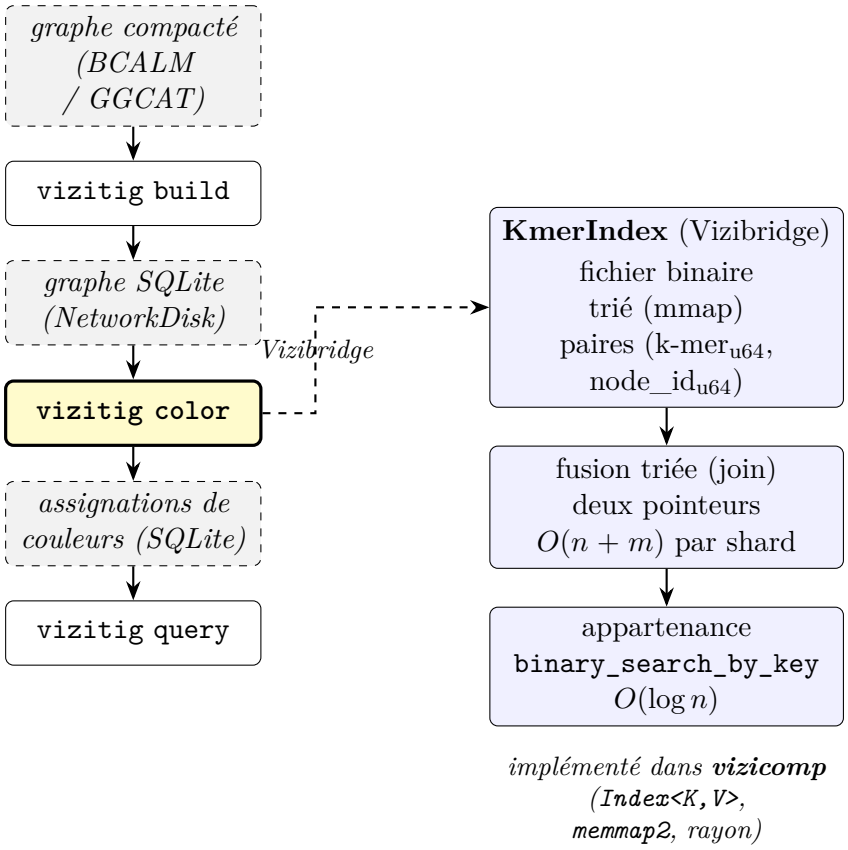

FIGURE 2 – Pipeline de Vizitig (gauche) et détail de la couche Vizibridge/vizicomp invoquée lors de l'étape `vizitig color` (droite).

À l'échelle d'un seul échantillon, la complexité $O(n+m)$ de la fusion triée est acceptable. Cependant, dans un contexte pangenomique impliquant des dizaines ou des centaines d'échantillons, cette opération est répétée pour chaque échantillon, ce qui en fait le principal goulot d'étranglement du pipeline.
Ce travail émet l'hypothèse que les Roaring Bitmaps – et plus précisément leur variante 64 bits (les k-mers pour $k = 31$ étant des entiers de 62 bits stockés dans un `u64`, la variante standard 32 bits est insuffisante) – pourraient offrir de meilleures performances que les fichiers binaires triés pour ces trois opérations fondamentales.

2 Roaring bitmaps

2.1 Approches précédentes de compression

Les Roaring Bitmaps sont apparus comme une réponse aux limites des formats de compression antérieurs. Pour comprendre pourquoi cette approche est bien adaptée au pro-

blème, examinons les principales méthodes de représentation d'ensembles d'entiers, en nous concentrant sur leur comportement face à des données de densité variable.
Le *Run-Length Encoding* (RLE) repose sur l'idée de remplacer les séquences de bits identiques par une représentation compacte indiquant la valeur et le nombre de répétitions. Par exemple, une séquence de sept zéros peut être représentée sous la forme 0×7. Cette approche permet une compression efficace lorsque les données contiennent de longues plages répétées, mais présente une limitation importante : l'accès aléatoire est coûteux, car il nécessite de décoder toutes les séquences précédentes.
Le format *Word-Aligned Hybrid* (WAH) [13] a été conçu pour dépasser cette limitation. Son principe repose sur l'alignement avec la taille du mot machine : les processeurs opèrent sur des mots (typiquement 32 bits) plutôt que sur des octets individuels, ce qui permet d'effectuer des opérations logiques bit-à-bit sur des blocs entiers en une seule instruction. Concrètement, le bitmap à compresser est partitionné en blocs de 31 bits (taille du mot moins un bit réservé au type). Les blocs contenant un mélange de 0 et de 1 – appelés blocs *sales* (*dirty*) – sont stockés comme des *mots littéraux* : le bit de poids fort vaut 1 et les 31 bits restants reproduisent le bloc tel quel. En revanche, les suites de blocs consécutifs contenant uniquement des 0 ou uniquement des 1 sont compressées en un seul *mot de remplissage* (*fill word*) : le bit de poids fort vaut 0, le bit suivant indique la valeur (0 ou 1), et les 30 bits restants encodent le nombre de blocs répétés. La figure 3 illustre ce mécanisme.

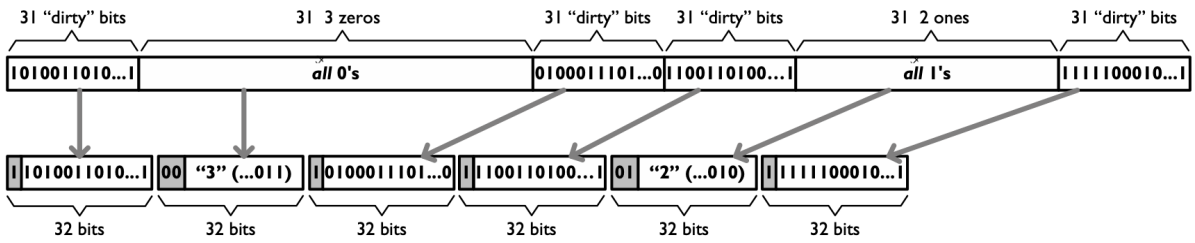

FIGURE 3 – Encodage WAH : les blocs sales sont stockés comme mots littéraux ; les plages homogènes sont compressées en mots de remplissage. Figure tirée de Colantonio et Di Pietro [14].

Cette organisation garantit que les opérations ensemblistes (ET, OU) peuvent être effectuées directement sur la représentation compressée : il suffit de traiter les mots littéraux bit-à-bit et de sauter les mots de remplissage en un seul pas, sans décompression préalable.
La structure *Concise* (Compressed 'N' Composable Integer Set) [14] ajoute une couche de compression supplémentaire : lorsque les données sont clairsemées, un bitmap non compressé gaspille de la mémoire en stockant un grand nombre de zéros ; Concise représente ces blocs creux plus efficacement en opérant au niveau des entiers plutôt que des mots. Cependant, ni WAH ni Concise ne résolvent le problème de l'accès aux éléments individuels : déterminer si un entier donné appartient à l'ensemble nécessite de parcourir et

décoder séquentiellement la structure, ce qui rend les tests d'appartenance coûteux.
Ces formats présentent donc deux limites fondamentales : ils supposent une densité uniforme (dense ou clairsemée) et ne permettent pas l'accès aléatoire efficace. C'est pour répondre à ces deux problèmes simultanément que les Roaring Bitmaps ont été proposés.

2.2 Principe des bitmaps

Dans le pipeline de colorisation de Vizitig, les k-mers sont encodés sous forme d'entiers (sur 62 bits pour $k = 31$, stockés dans un `u64`). Le problème se ramène donc à représenter et manipuler des ensembles d'entiers de grande taille – ce pour quoi les bitmaps constituent une solution naturelle.
Une représentation bitmap consiste à associer à chaque entier possible une position dans un vecteur de bits. Si l'élément appartient à l'ensemble, le bit correspondant est égal à 1, sinon il est égal à 0.
Cette structure peut être interprétée comme une fonction indicatrice. Pour représenter un ensemble d'entiers appartenant à l'intervalle $[0, n]$, un vecteur de $n + 1$ bits suffit : le bit à la position i vaut 1 si et seulement si i appartient à l'ensemble.
Par exemple, l'ensemble $\{2, 5, 7\}$ dans l'univers $[0, 7]$ est représenté par le vecteur de 8 bits suivant (le bit de poids faible correspond à la position 0) :

$$\underbrace{1}_{7}\ \underbrace{0}_{6}\ \underbrace{1}_{5}\ \underbrace{0}_{4}\ \underbrace{0}_{3}\ \underbrace{1}_{2}\ \underbrace{0}_{1}\ \underbrace{0}_{0}$$

L'avantage principal est que les opérations ensemblistes se réduisent à des opérations logiques bit-à-bit. L'intersection de $\{2, 5, 7\}$ et $\{1, 5, 7\}$ s'obtient par un ET bit-à-bit :

$$10100100\ \texttt{AND}\ 10100010 \ = \ 10100000 \quad \longrightarrow \quad \{5, 7\}$$

Sur les architectures modernes, une seule instruction SIMD peut traiter 256 ou 512 bits simultanément, ce qui rend ces opérations extrêmement rapides.
Cependant, lorsque l'univers des valeurs est très grand et que l'ensemble contient peu d'éléments, une représentation bitmap classique devient inefficace en termes d'espace mémoire.

2.3 Structure des Roaring bitmaps

Les Roaring Bitmaps [7] répondent aux deux limites identifiées précédemment – hypothèse de densité uniforme et absence d'accès aléatoire efficace – en introduisant un principe d'adaptation locale : plutôt que d'imposer une représentation uniforme, la structure choisit dynamiquement, pour chaque groupe d'entiers, le format le plus compact en fonction de leur densité.

Trois types de containers sont définis. Le *array container* stocke les valeurs sous forme de liste triée d'entiers 16 bits non signés ; il est adapté aux ensembles clairsemés. Le *bitmap container* utilise un bitmap plat de $2^{16} = 65\,536$ bits (1 024 mots de 64 bits) ; il est efficace lorsque les données sont denses. Le *run container*, introduit dans [12], stocke les séquences consécutives sous forme d'intervalles compressés ; il s'avère peu pertinent ici, les k-mers canoniques (le minimum lexicographique entre un k-mer et son complémentaire inverse) suivant une distribution pseudo-aléatoire sans longues plages d'entiers contigus.
Le choix entre array et bitmap container repose sur un seuil précis : 4 096 éléments. À ce point de transition, les deux représentations occupent exactement le même espace ($4\,096 \times 16$ bits $= 65\,536$ bits). En dessous du seuil, le tableau est plus compact ; au-dessus, le bitmap l'est. Ce mécanisme garantit qu'au niveau de chaque container, chaque entier occupe au plus 16 bits.
Organisation en deux niveaux. Pour organiser ces containers, les entiers sont partitionnés par leurs 16 bits de poids fort : chaque container regroupe les entiers partageant les mêmes 16 bits supérieurs et stocke leurs 16 bits inférieurs. Les containers sont organisés dans un tableau dynamique trié par ces bits supérieurs, qui constitue un *index de premier niveau*. Pour localiser le container d'un entier donné, il suffit d'extraire ses 16 bits supérieurs et d'effectuer une recherche dans ce tableau. Cet index reste typiquement très petit : pour $n = 1\,000\,000$ d'entiers, il contient au plus $\lceil 1\,000\,000/65\,536 \rceil = 16$ entrées, ce qui lui permet de résider entièrement dans le cache du processeur.

3 Application à Vizitig

3.1 L'index de k-mers actuel

L'index de k-mers utilisé par Vizitig est un fichier binaire projeté en mémoire (*memory-mapped file*). Il stocke des paires (k-mer, node_id), où le k-mer est l'une des sous-séquences appartenant à un unitig, et le `node_id` est l'identifiant de cet unitig dans le graphe compacté. La projection mémoire évite de charger l'intégralité du fichier en mémoire vive : les pages sont lues ou modifiées par fragments, à la demande du système d'exploitation. Grâce à un compteur de référence partagé (`Arc<Mmap>`), plusieurs fils d'exécution peuvent accéder simultanément à la même projection, ce qui permet de traiter les fragments (*shards*) en parallèle.
La construction de l'index se déroule en deux étapes : les entrées brutes sont d'abord sérialisées en binaire, puis triées en place par un tri rapide parallèle (`par_sort_unstable`, rayon), en $O(n \log n)$.
Sur cette structure, deux opérations sont exposées. La recherche d'appartenance s'effectue en $O(\log n)$ par requête (`binary_search_by_key`). La colorisation d'un échantillon repose sur une fusion triée à deux pointeurs entre l'index du graphe et l'ensemble des k-mers de

l'échantillon, en $O(n+m)$.
L'implémentation est en outre optimisée par l'utilisation d'instructions SIMD : Vizibridge détecte à l'exécution les extensions disponibles sur le processeur (AVX2, AVX512) et les utilise pour les opérations arithmétiques sur les k-mers. Ce point est structurant pour la comparaison : la référence de performance est déjà du Rust compilé avec SIMD. Une implémentation à base de Roaring Bitmaps devra donc également exploiter ces mêmes extensions pour permettre une comparaison équitable.

3.2 Analyse de la densité

Pour comprendre quels types de containers seront utilisés par les Roaring Bitmaps dans ce contexte, il est nécessaire d'analyser la densité des k-mers dans l'espace des entiers. Le type de container sélectionné détermine en effet le mode d'intersection utilisé – ET bit-à-bit pour les *bitmap containers*, recherche par bonds pour les *array containers* – et donc si l'avantage SIMD de Roaring s'applique.
Taille de l'univers. L'alphabet ADN comporte quatre lettres ; pour $k = 31$, cela donne $4^{31} \approx 4{,}6 \times 10^{18}$ k-mers possibles. Sous forme d'entiers en encodage 2 bits, ils s'inscrivent dans l'intervalle $[0,\ 2^{62})$.
Comportement de Roaring64. RoaringTreemap partitionne par les 32 bits de poids fort, produisant jusqu'à $2^{32} \approx 4 \times 10^9$ partitions possibles. Un *bitmap container* n'est sélectionné que lorsqu'une partition dépasse 4 096 éléments – ce seuil n'est atteint qu'à partir d'environ 10^{13} k-mers au total, bien au-delà de tout organisme ou collection pangenomique réaliste. À titre de comparaison :

Organisme	k-mers ($k = 31$)	Occ. moy. / partition
E. coli	4×10^6	0,001
Humain	3×10^9	0,7
Blé hexaploïde	$1{,}7 \times 10^{10}$	4
Paris japonica	$1{,}5 \times 10^{11}$	35
Pangenome (100 humains)	$\sim 10^{10}$	~ 2

Table 1 – Occupation moyenne par partition dans RoaringTreemap pour différents organismes ($k = 31$). *Paris japonica* est le plus grand génome connu.

Même le plus grand génome végétal connu affiche une occupation moyenne de 35 éléments par partition, très en dessous du seuil de 4 096. Les *array containers* dominent donc universellement ; les *bitmap containers* ne sont en pratique jamais sélectionnés. Les *run containers* sont également sans intérêt ici : les k-mers canoniques sont distribués pseudo-aléatoirement dans l'espace des entiers, sans longues plages d'entiers consécutifs. Cette dispersion est une conséquence directe de la composition nucléotidique des génomes réels : les 32 bits de poids fort d'un k-mer encodent ses 16 premières bases, et la distribution équilibrée des quatre nucléotides fait que la fenêtre glissante explore l'ensemble

des $2^{32} \approx 4 \times 10^9$ préfixes possibles, remplissant ainsi toutes les partitions de manière approximativement uniforme.

Le rôle de l'encodage. Les Roaring Bitmaps ont été conçus pour des ensembles denses d'identifiants entiers : identifiants de documents dans les moteurs de recherche, identifiants utilisateurs dans les bases OLAP, horodatages. Dans ces cas, les données s'inscrivent dans un intervalle dense $[0, N)$ et les *bitmap containers* dominent. Il serait possible de réencoder chaque k-mer comme un rang entier $0 \ldots N-1$ – Roaring verrait alors des données denses. L'encodage 2 bits est néanmoins conservé pour trois raisons : il ne nécessite aucune connaissance préalable de l'ensemble complet des k-mers, chaque conversion est un calcul direct sans table de correspondance, et il permet les opérations arithmétiques SIMD sur les k-mers (complément inverse, canonicalisation).

Conséquence. L'avantage SIMD de Roaring – le ET bit-à-bit sur les *bitmap containers* – ne s'applique pas au cas des k-mers. Avec des *array containers*, Roaring recourt à la recherche par bonds (*galloping search*), sans opération vectorielle en masse. La présente analyse ne permet donc pas de conclure que les Roaring Bitmaps surpassent la fusion triée combinée aux instructions SIMD de l'implémentation actuelle. La comparaison de performances est une question empirique, qui dépasse le cadre de ce travail.

3.3 Analyse des opérations fondamentales

Compte tenu de la description de l'index actuel (section 3.1) et de l'analyse de densité (section 3.2), nous examinons l'impact d'un remplacement par Roaring Bitmaps sur les trois opérations fondamentales définies en section 1.3.

Appartenance. Étant donné un k-mer, l'opération détermine s'il est présent dans l'index – c'est notamment le mécanisme utilisé lors des requêtes `Kmer(seq)`. L'implémentation actuelle repose sur une recherche dichotomique sur l'index décrit en section 3.1, ce qui donne une complexité $O(\log n)$ par requête, où n est le nombre de k-mers dans le fragment (*shard*) concerné.

Intersection. L'intersection constitue le cœur de l'étape `vizitig color` : il s'agit d'identifier les k-mers communs entre l'index du graphe et l'ensemble des k-mers d'un échantillon biologique. L'implémentation actuelle procède par fusion de deux listes triées (*sorted-merge join*), ce qui donne une complexité $O(n + m)$, où n est la taille de l'index et m celle de l'échantillon. Avec des *array containers* – le seul type sélectionné pour les k-mers, comme établi en section 3.2 – l'intersection Roaring s'effectue par recherche par bonds (*galloping search*), de complexité $O(p \log(q/p))$ pour deux ensembles de tailles $p \leq q$. Cette complexité est avantageuse lorsque les ensembles sont fortement asymétriques : dans `vizitig color`, l'index du graphe (q) est typiquement bien plus grand que l'ensemble des k-mers d'un échantillon individuel (p). Ce gain potentiel est particulièrement significatif dans un contexte pangenomique : une étude impliquant des dizaines voire des

centaines d'échantillons répète cette opération autant de fois, ce qui en fait une part dominante de la complexité totale du pipeline.

Jointure. Pour chaque k-mer identifié lors de l'intersection, la jointure récupère le `node_id` associé, afin de construire les assignations de couleurs. Dans l'implémentation actuelle, cette opération n'engendre aucun surcoût : chaque entrée de l'index stocke conjointement le k-mer et son `node_id` (`IndexEntry{key, val}`), de sorte que la fusion triée les produit simultanément. C'est cette propriété que toute alternative devra préserver ou compenser.

3.4 Proposition

Nous proposons de remplacer le KmerIndex par une représentation bitmap 64 bits, en utilisant RoaringTreemap (bibliothèque `roaring` en Rust) ou l'API 64 bits de CRoaring. La variante standard 32 bits est insuffisante : les k-mers pour $k = 31$ sont des entiers de 62 bits.

Appartenance. La vérification s'effectue via `contains(k_mer)` sur le bitmap. Le chemin emprunté passe par l'array container, qui effectue une recherche dichotomique en $O(\log n)$ – asymptotiquement identique à l'implémentation actuelle, mais avec un comportement potentiellement différent vis-à-vis du cache processeur.

Intersection. L'intersection entre le bitmap du graphe et le bitmap de l'échantillon s'effectue, avec des array containers, par une recherche par bonds (*galloping search*), de complexité $O(p \log(q/p))$ pour deux ensembles de tailles $p \leq q$. Cette complexité est avantageuse lorsque l'asymétrie est forte – ce qui est précisément le cas dans `vizitig color`, où l'index du graphe (q) est typiquement bien plus grand que l'ensemble des k-mers d'un échantillon individuel (p).

Jointure. Cette opération présente un défi structurel : un bitmap Roaring stocke uniquement les valeurs entières des k-mers, sans les `node_id` associés. Après l'intersection, une deuxième étape est nécessaire pour récupérer les identifiants de nœuds. Deux approches sont envisageables : maintenir un tableau trié séparé pour la correspondance k-mer $\rightarrow$ `node_id`, ou utiliser le bitmap uniquement pour les étapes d'appartenance et d'intersection et conserver le KmerIndex existant pour la jointure.

Ce travail émet l'hypothèse que Roaring64 peut améliorer les performances des opérations d'appartenance et d'intersection lors de l'étape `vizitig color`, et constitue le point de départ d'une validation empirique.

4 Conclusion

Nous avons étudié dans quelle mesure le format de données Roaring Bitmaps peut accélérer les opérations de colorisation dans Vizitig, notamment lors de l'étape `vizitig color` où la fusion triée entre l'index du graphe et les k-mers de chaque échantillon constitue le

principal goulot d'étranglement à grande échelle.
Nous avons constaté que l'avantage attendu ne se concrétise pas aussi simplement qu'espéré. L'encodage 2 bits des k-mers les disperse dans un univers de $4{,}6 \times 10^{18}$ valeurs, de sorte que les *bitmap containers* ne sont jamais sélectionnés et que l'avantage SIMD principal de Roaring – le ET bit-à-bit – ne s'applique pas. Un avantage plus limité reste néanmoins possible : la recherche par bonds (*galloping search*) sur les *array containers* offre une complexité $O(p \log(q/p))$ qui exploite l'asymétrie naturelle entre l'index du graphe et les ensembles de k-mers des échantillons individuels.
Nous identifions par ailleurs la jointure comme un problème ouvert : un bitmap Roaring ne stockant pas les `node_id` associés aux k-mers, toute implémentation devra résoudre ce défi structurel avant de pouvoir remplacer complètement l'index actuel. Une évaluation empirique des deux approches, dans des conditions SIMD équivalentes, permettrait d'établir les véritables ratios de performance et de déterminer si le gain sur l'appartenance et l'intersection justifie la complexité supplémentaire introduite par la gestion de la jointure.

Références

[1] Metzker, *Sequencing technologies – the next generation*, Nature Reviews Genetics, 2010. `https://doi.org/10.1038/nrg2626`

[2] Compeau, Pevzner et Tesler, *Why are de Bruijn graphs useful for genome assembly ?*, Nature Biotechnology, 2011. `https://pmc.ncbi.nlm.nih.gov/articles/PMC5531759/`

[3] Pevzner et al., A New Eulerian Path Approach to Fragment Assembly. `https://pmc.ncbi.nlm.nih.gov/articles/PMC55524`

[4] Chikhi et al., BCALM 2. `https://pubmed.ncbi.nlm.nih.gov/27307618/`

[5] Scalable, ultra-fast, and low-memory construction of compacted de Bruijn graphs with Cuttlefish 2 `https://pubmed.ncbi.nlm.nih.gov/36076275/`

[6] Vizitig. `https://www.biorxiv.org/content/10.1101/2025.04.19.649656v1`

[7] Lemire et al., Roaring Bitmaps. `https://arxiv.org/abs/1402.6407`

[8] Z. Iqbal, M. Caccamo, I. Turner, P. Flicek, and G. McVean. *De novo assembly and genotyping of variants using colored de Bruijn graphs.* Nature Genetics, 44(2), 2012.

[9] F. Almodaresi, P. Pandey, M. Ferdman, M. Patro, and R. Johnson. *An efficient, scalable and exact representation of high-dimensional color information enabled via de Bruijn graph search.* Bioinformatics, 34(13), 2018.

[10] M. D. Muggli, E. Bowe, J. Puglisi, et al. *Succinct colored de Bruijn graphs.* Bioinformatics, 33(20), 2017.

[11] A. Bowe, T. Onodera, K. Sadakane, and T. Shibuya. *Succinct de Bruijn Graphs*. Algorithms in Bioinformatics (WABI), 2012.

[12] D. Lemire, G. Ssi-Yan-Kai, and O. Kaser. *Consistently faster and smaller compressed bitmaps with Roaring*. arXiv preprint arXiv :1603.06549, 2018.

[13] K. Wu, E. J. Otoo, and A. Shoshani. *Optimizing bitmap indices with efficient compression*. ACM Transactions on Database Systems, 31(1) :1–38, 2006.

[14] A. Colantonio and R. Di Pietro. *Concise : Compressed 'N' Composable Integer Set*. Information Processing Letters, 110(16) :644–650, 2010. `https://doi.org/10.1016/j.ipl.2010.05.018`